

Applied Linear Algebra

Matrix Calculus for Learning Systems

Unit 3

Devi Prasad M
Manipal School of Information Sciences
Manipal

October 2026

- Derivatives in one dimension (0.5 h)
- Gradients in multiple dimensions (1 h)
- Chain rule in multiple dimensions (1 h)
- Gradients of matrix expressions (2 h)
- Optimization using gradient descent (3.5 h)
 - single-sample updates
 - Logistic regression (No hidden layer)
 - Softmax regression (No hidden layer)
- Backpropagation (3 h)
 - Computational graphs
 - Multi-layer perceptrons (Hidden Layers)

A derivative is the rate of change of a function with respect to changes in its inputs.

For functions $f : \mathbb{R} \rightarrow \mathbb{R}$, the derivative of f at x is defined as:

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

$f'(x)$ measures how fast the output changes as the input changes:

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

This limit tells us what the ratio between the perturbation h in the input and the change in the function value $f(x+h) - f(x)$ converges to as h shrinks to zero.

If the limit exists, we say that f is *differentiable* at x .

- **Local linearity.** Near any point, a smooth function behaves like a straight line:

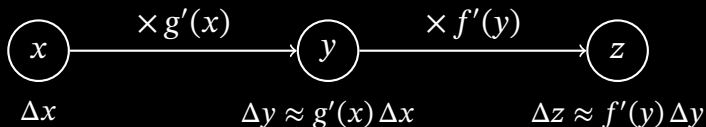
$$f(x + \Delta x) \approx f(x) + f'(x) \Delta x$$

- **Direction of descent.** To decrease f , move x against the sign of $f'(x)$.
- At a minimum $f'(x) = 0$, though not every point with $f'(x) = 0$ is a minimum.

Rates of change multiply through a composition:

$$\frac{d}{dx} f(g(x)) = f'(g(x)) g'(x)$$

Follow a small nudge. Write $y = g(x)$ and $z = f(y)$.



Substituting, $\Delta z \approx f'(y) g'(x) \Delta x$, so each stage scales the nudge and the scale factors multiply.

Example. If y changes $3\times$ as fast as x , and z changes $2\times$ as fast as y , then z changes $3 \times 2 = 6\times$ as fast as x .

Watch out. f' is evaluated at $y = g(x)$, where the nudge actually arrives, not at x .

<https://www.3blue1brown.com/lessons/chain-rule-and-product-rule/>

Basic Functions

- Exponential: $\exp(z)$
— e^z
- Natural Log: $\ln(z)$

Activation Functions

- Sigmoid: $\sigma(z) = \frac{1}{1 + e^{-z}}$
- Tanh: $\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$
- ReLU: $\max(0, z)$
- Leaky ReLU: $\max(0.01z, z)$
- GELU: $z \cdot \Phi(z)$

$$\begin{aligned}\frac{d}{dz}(e^z) &= \lim_{h \rightarrow 0} \frac{e^{z+h} - e^z}{h} && \text{(Limit definition)} \\ &= \lim_{h \rightarrow 0} \frac{e^z \cdot e^h - e^z}{h} && \text{(Exponent rules)} \\ &= \lim_{h \rightarrow 0} \frac{e^z(e^h - 1)}{h} && \text{(Factor out } e^z) \\ &= e^z \cdot \lim_{h \rightarrow 0} \frac{e^h - 1}{h} && \text{(Move } e^z \text{ outside the limit)} \\ &= e^z \cdot 1 && \text{(Using the identity } \lim_{h \rightarrow 0} \frac{e^h - 1}{h} = 1) \\ &= e^z\end{aligned}$$

Python snippet to verify $e^h \approx 1 + h$ for small h .

Derivative of the Natural Logarithm

Unit 3

$$\text{Let } y = \ln(z)$$

$$e^y = z \quad (\text{Rewrite in exponential form})$$

$$\frac{d}{dz}(e^y) = \frac{d}{dz}(z) \quad (\text{Differentiate both sides})$$

$$e^y \cdot \frac{dy}{dz} = 1 \quad (\text{Apply the chain rule})^\dagger$$

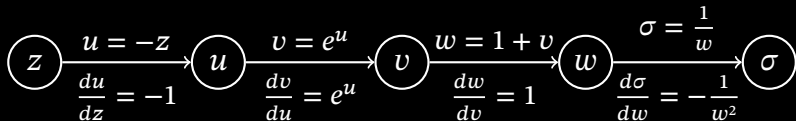
$$\frac{dy}{dz} = \frac{1}{e^y}$$

$$\frac{dy}{dz} = \frac{1}{z} \quad (\text{Substitute } e^y = z)$$

$^\dagger f(g(z)) = e^{g(z)}, g(z) = y.$

Derivative of the Sigmoid via the Chain Rule

Simplify $\sigma(z) = 1/(1 + e^{-z})$ into four steps. Operation is shown above the arrow, and the local derivative below the arrow.



Chain rule: multiply the local derivatives along the chain.

$$\frac{d\sigma}{dz} = \frac{d\sigma}{dw} \frac{dw}{dv} \frac{dv}{du} \frac{du}{dz} = \left(-\frac{1}{w^2}\right)(1)(e^u)(-1) = \frac{e^{-z}}{(1 + e^{-z})^2}$$

Rewrite in terms of σ :

$$\frac{e^{-z}}{(1 + e^{-z})^2} = \underbrace{\frac{1}{1 + e^{-z}}}_{\sigma(z)} \cdot \underbrace{\frac{e^{-z}}{1 + e^{-z}}}_{1 - \sigma(z)} \implies \boxed{\sigma'(z) = \sigma(z)(1 - \sigma(z))}$$

More About the Sigmoid

Show that $\sigma(-z) = 1 - \sigma(z)$

$$\text{Note } \sigma(z) = \frac{1}{1 + e^{-z}} \quad \text{and} \quad \sigma(-z) = \frac{1}{1 + e^z}$$

$$\begin{aligned} 1 - \sigma(z) &= 1 - \frac{1}{1 + e^{-z}} \\ &= \frac{e^{-z}}{1 + e^{-z}} = \frac{1}{e^z} \cdot \frac{1}{1 + e^{-z}} \\ &= \frac{1}{1 + e^z} \\ &= \sigma(-z) \end{aligned}$$

$$\begin{aligned}\frac{d}{dz}\sigma(-z) &= \frac{d}{dz}(1 - \sigma(z)) \\ &= 0 - \sigma'(z) \\ &= -\sigma(z)(1 - \sigma(z))\end{aligned}$$

- $\sigma'(-z)$: take σ' , then evaluate it at $-z$.

$$\sigma'(-z) = \sigma(-z)(1 - \sigma(-z)) > 0$$

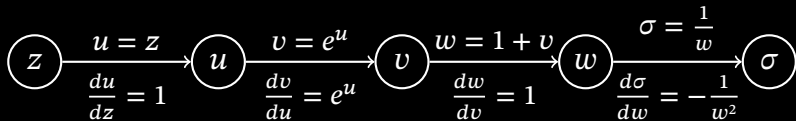
- $\frac{d}{dz}\sigma(-z)$: differentiate the composite with respect to z .

$$\frac{d}{dz}\sigma(-z) = -\sigma'(-z) < 0$$

The two differ by exactly the chain-rule factor $\frac{d}{dz}(-z) = -1$.

Derivative of $\sigma(-z)$ Using the Chain Rule

Simplify $\sigma(-z) = 1/(1 + e^z)$ into four steps.



Chain rule: multiply the local derivatives along the chain.

$$\frac{d\sigma}{dz} = \frac{d\sigma}{dw} \frac{dw}{dv} \frac{dv}{du} \frac{du}{dz} = \left(-\frac{1}{w^2}\right)(1)(e^u)(1) = -\frac{e^z}{(1 + e^z)^2}$$

Rewrite in terms of σ :

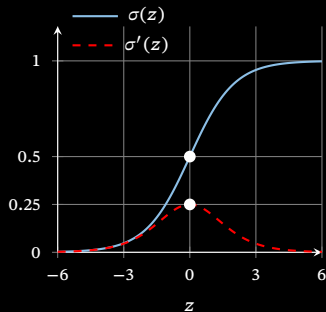
$$-\frac{e^z}{(1 + e^z)^2} = -\underbrace{\frac{1}{1 + e^z}}_{\sigma(-z)} \cdot \underbrace{\frac{e^z}{1 + e^z}}_{1 - \sigma(-z)} \implies \boxed{\sigma'(-z) = -\sigma(-z)(1 - \sigma(-z))}$$

Derivative of $\sigma(-z)$ – One More Time!

Show the two forms of $\frac{d}{dz}\sigma(-z)$ are equivalent:

$$\begin{aligned}\frac{d}{dz}\sigma(-z) &= -\sigma(z)(1 - \sigma(z)) \\ &= -(1 - \sigma(-z))\sigma(-z) \quad (\text{Use } \sigma(-z) = 1 - \sigma(z)) \\ &= -\sigma(-z)(1 - \sigma(-z)) \quad (\text{Rearrange})\end{aligned}$$

The Sigmoid and Its Derivative



- **Range.** $0 < \sigma(z) < 1$ and $\sigma(0) = \frac{1}{2}$, so the output reads as a probability.
- **Increasing.** $\sigma'(z) > 0$ for every z .
- **Bounded slope.**
 $\sigma'(z) \leq \frac{1}{4}$, largest at $z = 0$:
 $p(1 - p)$ peaks at $p = \frac{1}{2}$.
- **Symmetry.**
 $\sigma(-z) = 1 - \sigma(z)$, so
 $\sigma'(-z) = \sigma'(z)$.
- **Flat tails.** $\sigma'(z) \rightarrow 0$ as $|z| \rightarrow \infty$.

z	$\sigma(z)$	$\sigma'(z)$
0	0.5	0.25
2	0.881	0.105
4	0.982	0.018
6	0.998	0.0025
10	0.99995	0.000045

- **Saturation.** For large $|z|$ the output barely moves, so any gradient that passes through σ' is tiny and learning slows.
- **Deep networks.** Each sigmoid layer scales the backward signal by $\sigma' \leq \frac{1}{4}$, so it can shrink layer after layer.

The loss decides. For $p = \sigma(\mathbf{w}^T \mathbf{x})$:

- Cross-entropy: $\nabla_{\mathbf{w}} \ell = (p - y) \mathbf{x}$. The σ' factor cancels.
- Squared error: $\nabla_{\mathbf{w}} \ell = (p - y) p(1 - p) \mathbf{x}$. Nearly zero when the model is confidently wrong.

- **Partial derivative.** $\partial f / \partial x_j$: change one input, hold all the others fixed.
- **Gradient.** Collect the partials into a vector with the same shape as the input:
$$\nabla f(\mathbf{x}) = [\partial f / \partial x_1, \dots, \partial f / \partial x_n]^T.$$
- **Local linearity.** $f(\mathbf{x} + \Delta \mathbf{x}) \approx f(\mathbf{x}) + \nabla f(\mathbf{x})^T \Delta \mathbf{x}.$
- **Steepest ascent.** ∇f points in the direction of fastest increase and is perpendicular to the contour lines, so $-\nabla f$ is the steepest way downhill. At a minimum, $\nabla f = \mathbf{0}.$
- **Jacobian.** For $\mathbf{f} : \mathbb{R}^n \rightarrow \mathbb{R}^m$, the $m \times n$ matrix with entries $J_{ij} = \partial f_i / \partial x_j$ records how every output responds to every input.

- **Change flows through intermediate variables.** If $\mathbf{x} \rightarrow \mathbf{y} = \mathbf{g}(\mathbf{x}) \rightarrow z = f(\mathbf{y})$, a change in x_j reaches z through every y_i that depends on x_j .
- **Sum over paths.** $\frac{\partial z}{\partial x_j} = \sum_i \frac{\partial z}{\partial y_i} \frac{\partial y_i}{\partial x_j}$: multiply along each path, then add across paths.
- **Matrix form.** Jacobians multiply:
 $\mathbf{J}_{f \circ g}(\mathbf{x}) = \mathbf{J}_f(\mathbf{g}(\mathbf{x})) \mathbf{J}_g(\mathbf{x})$.
- **Scalar output.** When the final output is one number (a loss), $\nabla_{\mathbf{x}} z = \mathbf{J}_g(\mathbf{x})^\top \nabla_{\mathbf{y}} z$: the gradient is passed backward through the transposed Jacobian.

- **Shape rule.** The gradient of a scalar with respect to a vector or matrix has the same shape as that vector or matrix. Checking shapes catches most errors.
- **Method.** Write the expression with indices, differentiate a single entry, then reassemble the result in matrix form.
- **Standard results.** $\nabla_{\mathbf{x}}(\mathbf{a}^\top \mathbf{x}) = \mathbf{a}$,
 $\nabla_{\mathbf{x}}(\mathbf{x}^\top \mathbf{A} \mathbf{x}) = (\mathbf{A} + \mathbf{A}^\top) \mathbf{x}$, $\nabla_{\mathbf{x}} \|\mathbf{A} \mathbf{x} - \mathbf{b}\|^2 = 2\mathbf{A}^\top (\mathbf{A} \mathbf{x} - \mathbf{b})$.
- **Linear layer.** For $\mathbf{z} = \mathbf{W} \mathbf{x}$ and a scalar loss L :
 $\nabla_{\mathbf{x}} L = \mathbf{W}^\top \nabla_{\mathbf{z}} L$ and $\nabla_{\mathbf{W}} L = (\nabla_{\mathbf{z}} L) \mathbf{x}^\top$, an outer product.
- **Elementwise functions.** For $\mathbf{h} = \phi(\mathbf{z})$ applied entry by entry, the Jacobian is diagonal, so $\nabla_{\mathbf{z}} L = \nabla_{\mathbf{h}} L \odot \phi'(\mathbf{z})$.

- **Learning as minimization.** Training means choosing parameters θ that minimize the average loss
$$L(\theta) = \frac{1}{N} \sum_{n=1}^N \ell_n(\theta).$$
- **Update rule.** $\theta \leftarrow \theta - \eta \nabla L(\theta)$. By local linearity, a small step along $-\nabla L$ lowers the loss. If η is too small, progress is slow; if too large, the steps overshoot or diverge.
- **Single-sample updates.** $\theta \leftarrow \theta - \eta \nabla \ell_n(\theta)$ for a randomly chosen example n . Each step is cheap, and the gradient is noisy but correct on average.

- **Logistic regression.** $p = \sigma(\mathbf{w}^\top \mathbf{x} + b)$ with cross-entropy loss gives $\nabla_{\mathbf{w}} \ell = (p - y) \mathbf{x}$: prediction error times input.
- **Softmax regression.** $\mathbf{p} = \text{softmax}(\mathbf{W}\mathbf{x} + \mathbf{b})$ with cross-entropy loss gives $\nabla_{\mathbf{W}} \ell = (\mathbf{p} - \mathbf{y}) \mathbf{x}^\top$ for one-hot $\mathbf{y}$, the same pattern extended to K classes. Both losses are convex, so every local minimum is a global one.

- **Computational graph.** The model and loss are written as a chain of simple operations. Each node needs only its own local derivative.
- **Forward, then backward.** The forward pass computes and stores every intermediate value. The backward pass applies the chain rule from the loss toward the inputs, reusing those stored values.
- **Error signals.** Each node receives $\partial L / \partial(\text{node})$ from the nodes it feeds. When a value feeds several nodes, the incoming signals add.
- **Efficiency.** One backward pass gives the gradient for every parameter, at a cost comparable to one forward pass.

- **Multi-layer perceptron.** With $\mathbf{z}_1 = \mathbf{W}_1\mathbf{x} + \mathbf{b}_1$, $\mathbf{h} = \phi(\mathbf{z}_1)$, $\mathbf{p} = \text{softmax}(\mathbf{W}_2\mathbf{h} + \mathbf{b}_2)$:

$$\delta_2 = \mathbf{p} - \mathbf{y},$$

$$\nabla_{\mathbf{W}_2} L = \delta_2 \mathbf{h}^\top,$$

$$\delta_1 = (\mathbf{W}_2^\top \delta_2) \odot \phi'(\mathbf{z}_1),$$

$$\nabla_{\mathbf{W}_1} L = \delta_1 \mathbf{x}^\top$$

- **Non-convexity.** Hidden layers make the loss non-convex. Gradient descent reaches a local solution, and that solution depends on the initialization.

- Michael Nielsen, *Neural Networks and Deep Learning*
<http://neuralnetworksanddeeplearning.com/>
- François Chollet, *Deep Learning with Python*,
<https://www.manning.com/books/deep-learning-with-python>
- 3Blue1Brown
<https://www.3blue1brown.com/?topic=calculus>